Reporte de Auditoría de Seguridad — Práctica 1 UC3

> **Curso:** Ciberseguridad Aplicada · **Unidad de Competencia 3** · **ITSON Práctica auditada:** P1 — Seguridad en APIs (BOLA · Mass Assignment · Validación estricta) · **Equipo:** Equipo Hector — Hector Andres Lagarda Alcantar (204733) · **Versión del documento:** 1.0 · **Fecha:** 11/11/2026

Sección 1. Portada e identificación

Campo	Valor
Organización / curso	Ciberseguridad Aplicada — ITSON, Ingeniería en Software
Unidad de competencia	UC3: Seguridad de APIs y DevSecOps
Práctica auditada	P1 — Seguridad en APIs
Nombre del proyecto	pedidos-api (versiones vulnerable y segura)
Equipo auditor (integrantes y roles)	Hector Andres Lagarda Alcantar (204733): explotación, corrección, documentación y QA
Fecha de la auditoría	11 de noviembre de 2026
Versión del software auditado	`main` del repositorio https://github.com/hectorlagarda204733-dev/uc3-practica1-equipo-hector
Revisión y aprobación	Docente de la materia

Sección 2. Resumen ejecutivo

Se realizó una auditoría de seguridad sobre la API REST de gestión de pedidos (versión vulnerable provista por la cátedra) utilizando como metodología el catálogo **OWASP API Security Top 10 (2023)**. Se identificaron **5 hallazgos**: 3 críticos (BOLA de lectura, BOLA de escritura y Mass Assignment con auto-asignación de rol ADMIN) y 2 altos (exposición de contraseñas en respuestas y validación inadecuada de esquemas). Las vulnerabilidades fueron corregidas en la versión segura (verificación de propiedad, DTOs, rol controlado por el servidor, esquema estricto y Bean Validation) y verificadas mediante 9 pruebas automatizadas (`mvn test` en verde) y retest con Postman: todas las mitigaciones operan correctamente (403/401/400 y rol USER). El proyecto queda apto para su uso en un entorno controlado, pendiente de controles de disponibilidad (rate limiting) y autenticación real (JWT) que se abordan en prácticas posteriores.

Tabla obligatoria del resumen:

Severidad	Hallazgos al inicio	Corregidos y verificados	Pendientes
Crítica	3	3	0
Alta	2	2	0
Media	0	0	0
Baja	0	0	0
Total	**5**	**5**	**0**

Sección 3. Alcance

Ítem	Detalle
Componentes auditados	Endpoints REST (`/api/pedidos` , `/api/pedidos/{id}` , `/api/usuarios` , `/api/usuarios/{id}`), controladores, servicios, modelos JPA, DTOs, configuración de Spring Boot (`application.properties` , `data.sql`), repositorios y pruebas de seguridad.
Fuera del alcance	Infraestructura de red, rendimiento y carga, auditoría de la nube, contenedores.
Supuestos	La autenticación se simula con el encabezado `X-User-Id` (diseño didáctico de la práctica); los datos son ficticios (data.sql); la base es H2 en memoria.
Limitaciones	Sin Docker en el equipo de trabajo; el puerto 8080 estaba ocupado por otro servicio (GeoServer), por lo que la auditoría se ejecutó en `http://localhost:8081` .

Sección 4. Metodología

- **Marco de referencia:** OWASP API Security Top 10 (2023) — https://owasp.org/API-Security/
- **Herramientas utilizadas:** Postman (colecciones de explotación y validación provistas por la cátedra), JUnit 5 + Spring Boot Test + MockMvc (pruebas de seguridad), Maven (ejecución), Apache NetBeans 21 (IDE), H2 (base en memoria).
- **Proceso aplicado:** (1) inventario de endpoints; (2) ejecución de la versión vulnerable; (3) explotación manual de vectores BOLA / mass assignment / validación con Postman; (4) registro de hallazgos con evidencia (petición y respuesta completas); (5) estudio de la versión segura; (6) retest de los mismos ataques (403/401/400); (7) ejecución de las pruebas automatizadas (`mvn test`); (8) documentación de resultados en los tres reportes.

Sección 5. Inventario de activos y superficie de ataque

Endpoint	Método	Autenticación	Autorización (vulnerable → segura)	Datos manejados	Riesgo si se compromete
/api/pedidos	GET	X-User-Id (ignorado → aplicado)	ninguna → solo pedidos propios	Descripciones, montos, fechas, dueño	Exposición de datos de terceros
/api/pedidos/{id}	GET	X-User-Id (ignorado → aplicado)	ninguna → propiedad del recurso	Pedido + datos del dueño (password en la vulnerable)	BOLA de lectura
/api/pedidos	POST	X-User-Id (ignorado → aplicado)	dueño desde el JSON → dueño desde la sesión	Nuevo pedido	BOLA de escritura, fraude
/api/usuarios	GET	— (público en ambas)	ninguna → solo el propio usuario	Usuarios + password (vulnerable)	Exposición masiva de credenciales
/api/usuarios/{id}	GET	X-User-Id (ignorado → aplicado)	ninguna → solo el propio usuario	Usuario + password (vulnerable)	Exposición de credenciales ajenas
/api/usuarios	POST	— (público en ambas)	N/A → DTO + rol fijo USER	Nombre, email, password, rol	Mass Assignment → escalada a ADMIN

Datos sembrados (data.sql): Ana (id 1, pedidos 1-2) y Beto (id 2, pedido 3), contraseñas ficticias `ana12345` / `beto12345`.

Sección 6. Hallazgos (detalle completo)

ID	Título	Severidad	OWASP	Estado
AUD-P1-001 (= P1-001)	BOLA de lectura en `GET /api/pedidos/{id}`	Crítica	API1:2023	Corregido
AUD-P1-002 (= P1-002)	BOLA de escritura en `POST /api/pedidos`	Crítica	API1:2023	Corregido
AUD-P1-003 (= P1-003)	Mass Assignment (auto-rol ADMIN)	Crítica	API3:2023	Corregido
AUD-P1-004 (= P1-004)	Exposición de contraseñas en respuestas	Alta	API3:2023 / API8:2023	Corregido
AUD-P1-005 (= P1-005)	Esquema de entrada sin validar	Alta	API8:2023	Corregido

El detalle completo de cada hallazgo (descripción, evidencia, impacto, facilidad y recomendación) se encuentra en el **Reporte de Vulnerabilidades** (`reportes/reporte_vulnerabilidades.md`), con los mismos identificadores. Evidencias: carpeta `evidencias/explotacion/` (capturas 00–06 de Postman con petición y respuesta completas).

Sección 7. Plan de remediación

ID	Remediación aplicada	Responsable	Archivos	Fecha de corrección	Evidencia
AUD-P1-001	Verificación de propiedad en servicio + 403/401	Hector Andres Lagarda Alcantar (204733)	`PedidoService.java`, `PedidoController.java`, `GlobalExceptionHandler.java`	11/11/2026	`PruebaBolaTest` + `evidencias/validacion/01` y `02`
AUD-P1-002	Dueño desde `X-User-Id`; DTO sin `usuario`	Hector Andres Lagarda Alcantar (204733)	`PedidoCreacionDTO.java`, `PedidoService.java`, `PedidoController.java`	11/11/2026	`evidencias/validacion/04`
AUD-P1-003	DTO de registro sin `id`, `rol` WRITE_ONLY, rol fijo USER	Hector Andres Lagarda Alcantar (204733)	`UsuarioRegistroDTO.java`, `UsuarioController.java`	11/11/2026	`PruebaMassAssignmentTest` + `evidencias/validacion/05`
AUD-P1-004	`@JsonIgnore` en password + DTOs de salida	Hector Andres Lagarda Alcantar (204733)	`Usuario.java`, `UsuarioRespuestaDTO.java`, `PedidoRespuestaDTO.java`	11/11/2026	`evidencias/validacion/00`
AUD-P1-005	`spring-boot-starter-validation` + `@Valid` + handler 400 por campo	Hector Andres Lagarda Alcantar (204733)	`pom.xml`, DTOs, `GlobalExceptionHandler.java`	11/11/2026	`PruebaValidacionTest` + `evidencias/validacion/06` y `07`

Sección 8. Verificación post-corrección (retest)

ID	Prueba de verificación	Resultado esperado	Resultado obtenido
AUD-P1-001	`PruebaBolaTest.usuario1_solicitaPedidoDeBeto_retorna403` + Postman (usuario 1 → pedido 3)	403 Forbidden	403 Forbidden ✓
AUD-P1-001b	`PruebaBolaTest.sinEncabezadoXUserId_retorna401` + Postman (sin encabezado)	401 Unauthorized	401 Unauthorized ✓
AUD-P1-002	Postman: POST con `"usuario":{"id":2}`	400 (campo desconocido)	400 Bad Request ✓
AUD-P1-003	`PruebaMassAssignmentTest.rolAdminEnJson_seIgnoraYSeAlmacenaUSER` + Postman	rol "ADMIN" ignorado, almacenado USER	201 con `"rol":"USER"` ✓
AUD-P1-003b	`PruebaMassAssignmentTest.campoDesconocidoEnJson_retorna400`	400 Bad Request	400 ✓
AUD-P1-004	`GET /api/pedidos/1` con X-User-Id: 1	Respuesta sin `password`	Sin `password` (DTO) ✓
AUD-P1-005	`PruebaValidacionTest` (email, password, nombre) + Postman 06/07	400 con mensaje por campo	400 con mensajes ✓
Suite completa	`mvn test` (NetBeans: Test Project)	9 pruebas, 0 fallos	`9 tests passed` ✓ (capturas 08/09)

Sección 9. Conclusiones y lecciones aprendidas

- **Estado final:** la API corregida cumple los controles evaluados en esta práctica: BOLA mitigada (403/401), Mass Assignment neutralizado (rol USER), contraseñas fuera de las respuestas y entradas validadas con 400 por campo. Suite de 9 pruebas de seguridad en verde.
- **Riesgos residuales:** la autenticación es simulada (`X-User-Id`, confiable solo dentro del laboratorio); no existe rate limiting (disponibilidad, práctica P2); las contraseñas de prueba son ficticias y los secretos reales deberían vivir en variables de entorno; el entorno de producción queda fuera de alcance.
- **Lecciones del equipo:** 1. BOLA ocurre cuando el servidor confía en el identificador que llega en la URL: la autorización debe compararse contra la sesión, no contra la petición. 2. Enlazar la entidad JPA directamente al JSON abre la puerta a Mass Assignment: usar DTOs de entrada/salida por defecto. 3. Los datos sensibles se filtran por diseño (DTOs + `@JsonIgnore`), no por "confiar" en que el cliente no los mire. 4. La validación de esquema y las pruebas de seguridad automatizadas convierten cada corrección en algo verificable en segundos.

Sección 10. Anexos

Anexo	Contenido
A	Capturas de explotación (ANTES) — `evidencias/explotacion/00` a `06` (petición y respuesta completas)
B	Capturas del retest (DESPUÉS) — `evidencias/validacion/00` a `07` (mismas peticiones, respuestas bloqueadas)
C	Listado de código: fragmentos vulnerable vs. corregido por hallazgo — Reporte de Correcciones, sección "Detalle de la corrección"
D	Salida de `mvn test` en verde — `evidencias/validacion/08_mvn_test_verde_netbeans.png` (Test Results: 9 tests passed) y `09_output_test_netbeans.png`
E	gitleaks / Dependency-Check — no aplica en P1 (se aborda en P3)
F	Configuración del repositorio — no aplica en P1 (se aborda en P4)
G	CI/SAST — no aplica en P1 (se aborda en P5)
H	Guía de ejecución: cómo reproducir esta auditoría — `README.md` del repositorio + `evidencias/bitacora.md`

Criterios de aceptación del documento

1. Sin texto de ejemplo de la plantilla. ✓
2. Cada hallazgo tiene evidencia verificable y referencia OWASP. ✓
3. Correspondencia exacta entre hallazgos, correcciones, verificaciones (mismos IDs). ✓
4. La sección de riesgos residuales es honesta. ✓
5. Anexos completos y legibles en el repositorio del equipo. ✓